Test audit of the herding model

02-120 final project (Lux–Marchesi herding model) · audit of 9 November 2026 · code at commit `ba6b062`

1. Summary

277,341	562	9	9
automatic checks on random inputs	existing tests: all pass	scripts run end to end	distinct issues found

Verdict. The simulation itself (`model.py`) passed every one of its checks: 6,000 random markets, 800 realistic runs per fitted setting, the switching step against its exact probability law, and "two runs must be identical" tests. A separate line-by-line comparison found its equations match `docs/SPEC.md`. All of the project's existing tests pass (562 passed in 95.45s (0:01:35)).

Before the fixes, the failed checks traced back to **9 distinct issues** (section 3). After the fixes, 0 checks fail. One was worth fixing before the web app (statistics on a constant series returned wrong numbers without an error). The others were input checks that let odd values (complex numbers, NaN, infinite prices) through, and documentation that had drifted from the code. **All 9 have now been fixed**, each with a test that failed first; every count in this report is from a rerun after the fixes. **No issue changes any figure or number in RESULTS.md**: each needs an input the project's real data and fitted settings never produce. Section 3 lists each fix and the test that now guards it.

> **Words used in this report. Oracle:** an independent way to get the right answer (scipy, a brute-force loop, or the rule written out again by hand), used to check the code's answer. **Randomized testing (fuzzing):** feeding a function thousands of random inputs, including strange ones, and checking every answer against the oracle. **Invariant:** something that must always be true, whatever the input (e.g. trader counts are whole numbers). **Metamorphic test:** two runs that must give identical results (e.g. a circuit breaker that can never fire must not change the run). **Edge case:** an unusual input at the boundary of what a function accepts (empty, constant, NaN, huge).

2. How the testing was done

2.1 Four layers

Layer	What it does	Checks	Failed
Existing suite (`tests/`)	The project's own tests (401 before the audit, the rest added with the fixes)	562	0
Sweep 1 (`scratch/test_audit/fuzz.py`)	Every public function, random and strange inputs, oracle for each answer (733 failures before the fixes)	268,541	2
Sweep 2 (`scratch/test_audit/fuzz2.py`)	Realistic runs with both fitted settings: identical-run tests, exact switching law, no-chartist bell curve	8,800	0
Scripts and spec	Every script run end to end; SPEC.md compared line by line with model.py	9 scripts	see 4, 5

Sweep 1 runs in about 17 seconds and sweep 2 in about one minute (8 cores). Both use fixed seeds, so they give the same result every time.

2.2 Results by module (sweep 1)

Module	Passed	Failed	Failures come from
`calibrate`	16,413	0	none
`experiments`	12,009	0	none
`kirman`	9,217	0	none
`market_data`	39	0	none
`model`	135,792	0	none
`policy`	7,011	0	none
`politics`	4,013	0	none
`stats`	84,045	0	none

2.3 Audit mistakes, caught and corrected

A test can be wrong too, so every failure was traced to its cause before it was counted as a bug. Three were the audit's own mistakes:

- Returns of size 1e-160 lost precision in the *oracle*: their squares are too small for a computer to store exactly. Real returns are never that small, so the case was changed to 1e-100.
- The shift-invariance test for kurtosis added offsets so large that the returns themselves were rounded away. The offsets now scale with the data.
- Two `acf` "mismatches" are cases where one lag window is constant. There the correlation is undefined: numpy gives NaN, and the code gives about 0. Neither answer is wrong, so these are not counted.

3. Issues found

Severity: **medium** = gives a wrong answer without an error; **low** = only on odd input, or a confusing message; **docs** = text and code disagree, the code is right. None of these affects a current figure or result.

Status: all 9 issues fixed on 9 November 2026. Each fix came after a test that failed on the old code; the issue descriptions below show how the code behaved before.

Issue	Fix	Test that now guards it
1 constant series	`stats._varying`: constant = largest equals smallest	`test_stats.py::test_every_spread_measure_rejects_any_constant` (84 cases)
2 complex numbers	`model._is_number` names the real types; `check` uses it	`test_model.py::test_complex_settings_are_rejected` (+ breakers, shocks)
3 NaN answers	`experiments.finite_1d`, used by `percentile`, `worst_fall`, `policy._paired`	`test_experiments.py`, `test_policy.py`: `*_rejects_nan*`
4 broken price files	`load_prices` checks columns, numbers, finite, positive (one message each)	`test_market_data.py::test_load_prices_explains_broken_files`
5 TypeErrors	`kirman.check` and `check_breakers` check types first	`test_kirman.py::test_N_that_is_not_a_number...`, `test_model.py::test_breaker_levels_that_are_not_a_list...`
6 empty DFA windows	`dfa` needs at least one window	`test_stats.py::test_dfa_needs_at_least_one_window_size`
7 settings name, CLI	`settings_path` rejects unknown kinds; new `parse_args` refuses `--draws` with `--refine`	`test_calibrate.py::test_load_rejects_an_unknown_kind...`, `test_cli_*`
8 docs	SPEC.md (trend at start, U cap, runaway rule), model.py comments; numba speed-up measured: about 30x	(text)
9 direct tests	tests call `check_breakers` and `check_shocks` directly, valid input included	`test_model.py::test_check_breakers_and_check_shocks_accept_valid_rules`

One slip during the fixing, caught and undone: the first version of the command-line test really ran `calibrate spx --refine` on the old code, which re-saved `settings/sp500_calibrated.json`. The fitted numbers were identical, but the file's explanatory note was dropped. The committed file was restored, and the test now checks the arguments through `parse_args`, without ever starting a fit.

Issue 1 medium FIXED Statistics on a constant series gave wrong numbers instead of an error

Fixed on 9 November 2026. A new helper, `stats._varying`, rejects a series whose largest and smallest values are equal. That compares two stored numbers, so rounding cannot hide a constant series. `standardize` and `dfa` use it, which covers every function below. New tests in `tests/test_stats.py`: `test_every_spread_measure_rejects_any_constant` (84 cases: 4 awkward constants × 3 lengths × 7 functions; 64 of them failed before the fix) and `test_tiny_but_varying_returns_are_still_measured`. The full suite passes (486 tests), and the sweep below was rerun after the fix: the 378 failures from this issue are gone. The description below is how the code behaved before the fix.

Where: `herding/stats.py:24` (`standardize`), and through it `kurtosis`, `acf`, `hurst`, `ccdf`, `fingerprint`.

What happens: The stats module promises to reject constant input ("no spread to measure"). It tests for that with `sd == 0`. But for a constant series of a nonzero number, rounding makes the computed standard deviation tiny instead of exactly 0. The check is skipped, and the code divides rounding noise by rounding noise:

```
c = np.full(1000, 0.0123)    # every day's return the same
c.std()                      # 1.7e-18, not 0
stats.kurtosis(c)            # -2.0     (should be an error)
stats.hurst(c, (16,32,64))   # -0.47    (should be an error; H cannot be negative)
stats.acf(c, [1])            # [nan]
```

Found by 378 failing checks in sweep 1. An all-zero series is still rejected correctly, because there the spread really is exactly 0.

Why it matters: Today it never happens: no real or simulated return series is constant. But the planned web app lets users choose settings. A setting that makes the price drift by exactly the same amount every day would show a kurtosis of -2.0 with no warning.

Suggested fix (for you to try): decide "constant" from the data itself, not from the rounded spread. For example, `if np.ptp(r) == 0 or sd <= 1e-12 * np.abs(r).max()`. Then add a test with `np.full(1000, 0.0123)`.

Issue 2 low FIXED Settings checks accept complex numbers

Where: `herding/model.py:89` (`check`) and `model.py:331` (`_is_number`, used by `check_breakers` and `check_shocks`).

What happens: Both test `isinstance(v, np.number)`. Numpy counts complex numbers as numbers, so `sigma_mu = 0.05+0.3j` passes `check`. `simulate` then silently drops the imaginary part. When `dt` is complex, `check` crashes with a `TypeError` instead of a clear `ValueError`. Sweep 1: 286 + 40 + 14 + 1 failing checks, all from this one cause.

Suggested fix: also reject `np.complexfloating` and `complex`. For example: `isinstance(v, (complex, np.complexfloating))` → error.

Issue 3 low FIXED Four helpers turn NaN into a silent answer

Where: `experiments.percentile` (`experiments.py:45`), `experiments.worst_fall` (`:54`), `policy.median_overshoot` (`policy.py:124`), `policy.paired_difference` (`:137`).

What happens: `percentile(nan, [1,2,3])` returns **0.0**, which reads as "the S&P is below every model history". `worst_fall([nan, 1.0], 1)` returns nan. The two bootstrap helpers return `(nan, nan, nan)`. The stats module rejects NaN everywhere, so these four are the odd ones out. A NaN could reach them if a fingerprint measure ever came out NaN (see issue 1).

Suggested fix: reuse `stats._returns`-style checking (finite, 1-D) at the top of each.

Issue 4 low FIXED `load_prices` lets an infinite price through, and two broken files give unclear errors

Where: `herding/market_data.py:34–35`.

What happens: A Close of `inf` passes the check `close > 0`, so the returns contain inf. That would later be rejected by `stats`, but far from the cause. A file with text in the Close column raises `TypeError: '>' not supported between 'str' and 'int'`. A file without a Close column raises a bare `KeyError: 'Close'`. The other 7 kinds of broken file tested (missing, NaN, negative, zero, unsorted, duplicate dates, bad date) all fail with clear messages, as intended.

Suggested fix: check `np.isfinite(close)`, check that Close is numeric, and check that the column exists, each with a `ValueError` naming the file.

Issue 5 low FIXED Two bad inputs give a `TypeError` instead of a clear message

`kirman.check("10", 0.1, 0.5)` (N given as text) fails inside `np.isfinite` (`kirman.py:26`). `simulate(..., breakers=dict(levels=0.1, halt_minutes=5))` fails at `tuple(breakers["levels"])` (`model.py:296`) with "'float' object is not iterable". Every other bad argument tested (24 for `simulate`, 11 for Kirman, 7 for `kirman.simulate`) gives a clear `ValueError`.

Issue 6 low FIXED `stats.dfa` with no window sizes returns empty arrays

`stats.dfa(x, ())` returns two empty arrays instead of an error (`stats.py:112`). `hurst` already guards against this (it needs at least 2 sizes), so only direct callers of `dfa`, such as `plots.dfa`, are exposed.

Issue 7 low FIXED Wrong advice for an unknown settings name

`python3 scripts/analysis/time_step_check.py nasdaq` fails with "settings/nasdaq.json not found; run: python3 -m herding.calibrate nasdaq". That command only prints the usage line, because the only kinds are `paper` and `spx`. The cause is `calibrate.settings_path` (`calibrate.py:36`), which invents a file name for any kind. Also, `--draws` is silently ignored when combined with `--refine` (`calibrate.py:405`).

Suggested fix: make `load` reject a kind other than paper/spx with a `ValueError` listing the two choices.

Issue 8 docs FIXED SPEC.md and comments in model.py that do not match the code

The model's equations agree with SPEC.md (section 4.2 lists what was compared). These are differences in wording; the code is right in each case:

- **SPEC.md:88** says the trend is 0 for the first 0.1 day. In the code (`model.py:198`) the trend window starts filled with the starting price, so the trend becomes nonzero as soon as the price moves. This affects only the first 10 time steps of the 1,000-day warm-up, which is thrown away.
- The cap of ±30 on the exponents U1, U21 and U22 (`model.py:56, 150`) is not in SPEC.md. It only acts in extreme states.
- "More than 1015 ticks in one step" also counts as a runaway (`model.py:243`). SPEC.md's runaway definition does not mention it.
- SPEC.md says a runaway is "ever more than e20 away", but the code checks only at each day's close (`model.py:269`).
- The comment at `model.py:237–238` says the code gives d(ln p)/dt = β(ED + μ). It actually gives this for dp/p. The log-price change is smaller by about tick/2 per tick.
- The `RunawayError` docstring (`model.py:74`) says "(all traders became chartists)", but the rule never looks at the chartist share.
- `model.py:24` says the code runs about 50x slower without numba, while `requirements.txt` says numba makes it about 20x faster. One of the two numbers is out of date.

Issue 9 low FIXED Untested functions in `tests/`

No test in `tests/` names `check_breakers`, `check_shocks` or `refine_saved` directly. The first two are reached through `simulate`, so they are tested indirectly. This audit's sweeps now cover them: 8,000 and 6,000 random cases, which is how issue 2 was found. `refine_saved` runs a full fit and remains untested.

4. Every feature, tried

Each feature of the project, what was tried, and what came out. "Exact" means the answer is known in advance and the code must match it to the last digit.

Feature	What was tried	Result
Market simulation (`model.simulate`)	6,000 random markets: 5 crowd sizes, 4 time steps, 3 tick sizes, every fitted setting drawn at random, 1–120 days, with and without chartists, breakers, shocks	Never crashed. Every run either finished or stopped with a proper `RunawayError`. Trader counts were whole, the floor always held, the opinion stayed in [-1, 1], the same seed gave an identical run, and returns equal the price changes.
Switching step (`_leave`)	2,000 random (group size, two rates, time step), each sampled 4,000 times	Matches the exact probability law (binomial, then split by rate) in all 2,000 cases
No-chartist check (proposal)	800 runs of 1,500 days; plus 500 runs with no noise and no news	Bell-curve returns in all 800 runs (|excess kurtosis| < 0.6). With no noise and no news, the price stays at exactly 1 (exact). `main.py`: kurtosis 0.038 over 19,329 days.
Circuit breakers	Random levels, pause lengths and cutoff minutes; breakers set too high to fire; US rules scaled to the model	A day closed at level 3 really closed 20%+ down. A day without a halt never closed below level 1. Breakers that cannot fire leave the run identical (exact). With breakers on, the run is identical to the run without them until the first halt (exact), which is what Fig 11's paired design relies on.
Policy surprises (shocks)	Random shock days and sizes, with the news switched off	The value moves by exactly log(1 + shock) on each shock day and by nothing else (exact). Shocks of 0% leave the run identical.
Runaway guard	`max_gap=None` vs default on 800 realistic runs; 100,000-day stability runs	Identical when nothing runs away. Long runs: 0 of 40 (paper-matched) and 2 of 40 (S&P-calibrated) ran away, as documented.
Numba speed-up	The same 60 seeds with and without numba	Identical results
Statistics (`stats`)	4,000 random series (Gaussian, fat-tailed, Cauchy, discrete with ties, constant, tiny), 800 DFA cases, plus known-answer tests	Kurtosis = scipy's; Hill = the textbook formula; DFA = a slow loop; scale and shift do not change the answers. White noise gives H ≈ 0.5, a random walk gives H ≈ 1.5, and an exact power law gives back its exponent (200 of 200). Issue 1: constant series.
Kirman's ant model	2,000 random (N, eps, delta); 30 long simulations of 400,000 steps	The exact solution is unchanged by one step of the chain, sums to 1, and is symmetric. The herding and no-herding regimes are on the side of Kirman's threshold the theory says. Long simulations visit each state as often as the exact answer predicts.
Calibration pieces	4,000 random draws; vectors with NaN and huge values; loss functions; resuming after a stop with a line cut off mid-write	Draws stay in range and pass `check`. NaN is clipped back in. The losses match their formulas. Saved scores replay without being recomputed, and a cut-off line does not corrupt the file.
Ensembles, percentiles	5,000 percentile cases vs scipy; 5,000 worst-fall cases vs brute force; 2,000 "typical history" cases	All match. Issue 3: NaN input.
Political parties	3,000 random date sets; inauguration-day boundaries; 500 party-gap cases vs a direct computation	All match, including the day a president takes office
Market data	All 3 index files, all 20 stocks, 10 kinds of broken file	Real files: sorted, finite, ending 2026-10-30. Black Monday is -20.5%. Issue 4: broken files.

Command line	`herding.calibrate` with no kind, an unknown kind, or a missing, zero, text or negative `--draws`; `time_step_check.py nasdaq`	All rejected with the usage line. Issue 7: the advice given for an unknown kind.

4.2 Spec vs code (SPEC.md compared with model.py, line by line)

Agree exactly: mood switching (sec. 1); strategy switching, including which group size multiplies which rate and the signs of U21/U22 (sec. 2); excess demand, Poisson ticks and the noise scaling with dt (sec. 3); news (sec. 4); floors and the order of updates within a step (sec. 5); the runaway rule; step_pull; parameter set I (N·tc = 10, N·γ = 5); and the US circuit-breaker rules (7% and 13% pause for 15 minutes only before 3:25 pm; 20% closes the day). The differences found are all in wording: see issue 8.

4.3 Numbers the scripts printed, compared with the documentation

Every number checked against RESULTS.md matched. `main.py`: H of returns 0.39 vs the S&P's 0.50, already listed as the model's clearest miss. Fig 10: below a1 ≈ 0.8 the S&P-calibrated model mostly runs away (1 of 12 histories survive at a1 0.4–0.6, 5 of 12 at 0.8), as RESULTS.md says. `time_step_check.py paper`: the paper-matched kurtosis falls from 29 to 3.4 as dt goes from 0.01 to 0.002, the documented time-step limitation. The S&P-calibrated settings stay steady (kurtosis 5.2, 7.0, 4.9).

4.1 Scripts run end to end

Command	Result	Time
`scripts/main.py`	ran cleanly	0 min 6 s
`scripts/make_figures.py`	ran cleanly	6 min 30 s
`scripts/analysis/efficient_market_claims.py`	ran cleanly	0 min 8 s
`scripts/analysis/bounce_back_after_big_days.py`	ran cleanly	0 min 3 s
`scripts/analysis/president_party_gap.py`	ran cleanly	0 min 9 s
`scripts/analysis/president_terms.py`	ran cleanly	0 min 6 s
`scripts/analysis/long_run_stability.py`	ran cleanly	0 min 45 s
`scripts/analysis/time_step_check.py spx`	ran cleanly	0 min 12 s
`scripts/analysis/time_step_check.py paper`	ran cleanly	0 min 13 s

Figure files afterwards (git status of `figures/`): unchanged, byte for byte: every figure was reproduced exactly from the committed code and settings.

5. Every check, with its count

Each row is one property, tested on many random inputs. Shaded rows have failures, explained in section 3.

Sweep 1: every function

Property checked	Passed	Failed
acf equals the Pearson correlation of r_t and r_t+lag (audit's own oracle; see 2.3)	2,140	2
Fit refuses an unknown kind	1	0
RunawayError.day is within the run	1,319	0
SavedLoss keeps working after a cut-off line (the next line is not glued on)	1	0
SavedLoss replays saved scores after a stop, without recomputing	1	0
a day closed by breaker level 3 closes at least that far down	1,860	0
a runaway scores RUNAWAY_LOSS	1	0
aggregate sums blocks of tau days	2,845	0
alpha_regression does not change when returns are rescaled	1,183	0
an un-halted day (pauses allowed all day) closes above level 1	871	0
black monday is -20.5% in the S&P file	1	0
block_bootstrap gives `draws` fingerprints	4	0
block_bootstrap refuses block outside 1..n	2	0
bootstrap helpers refuse NaN instead of returning NaN	2	0
bootstrap helpers refuse arrays of different length with ValueError	2	0
bootstrap helpers refuse fewer than 2 values	2	0
ccdf share = fraction of days with |z| >= x (ties: counted at the first)	2,845	0
ccdf: moves sorted, shares from 1 down to 1/n	2,845	0
chartist counts are whole numbers	4,681	0
check_breakers accepts exactly the valid rules	8,000	0
check_breakers raises only ValueError	8,000	0
check_shocks accepts exactly the valid shocks	6,000	0
check_shocks raises only ValueError	6,000	0
checkpointed resumes after a stop and returns every result in order	1	0
crash_measures: worst falls and crash-day count match brute force	2,000	0
democrat() matches a date-by-date lookup of the inauguration list	3,000	0
democrat() of no dates is empty	1	0
democrat() on inauguration-day boundaries	7	0
democrat() refuses dates before 1923-08-02	1	0
dfa equals a window-by-window loop	800	0
dfa_scales: increasing sizes from 16 to n/4, usable by dfa	2,000	0
draw: every fitted setting inside its range, and model.check accepts it	4,000	0
every stock file loads with finite returns	20	0
everything finite	4,681	0
floor: at least `floor` chartists and fundamentalists every day	3,880	0

from_vector clips any vector back into the ranges	4,000	0
from_vector never lets NaN through (Nelder-Mead can propose it)	4,000	0
from_vector(to_vector(p)) gives p back	4,000	0
halts only take values 0-3	4,681	0
herding regime (eps << (1-delta)/(N-1)): the extremes are the most likely	787	0
hill_alpha does not change when returns are rescaled	1,728	0
hill_alpha equals the textbook Hill formula	1,728	0
hill_alpha recovers the exponent of an exact power law (within 10%)	200	0
hill_alpha refuses too few / constant tail values	1,117	0
hurst of a random walk is about 1.5 (1.35-1.65)	60	0
hurst of white noise is about 0.5 (0.4-0.6)	60	0
kirman.check rejects bad parameters with ValueError	11	0
kirman.simulate rejects bad steps/k0 with ValueError	7	0
kirman.simulate visits each k as often as the exact answer says (total variation < 0.05)	30	0
kirman.simulate: length steps+1, starts at k0, stays in 0..N, moves by at most 1	600	0
kirman.stationary accepts exactly the parameters whose chances are valid	2,000	0
kirman.stationary is symmetric (A and B identical)	1,929	0
kirman.stationary is unchanged by one step of the chain (pi P = pi)	1,929	0
kirman.stationary sums to 1, nothing negative	1,929	0
kurtosis does not change when returns are shifted or rescaled	2,845	0
kurtosis equals scipy's excess kurtosis	2,845	0
kurtosis of Gaussian returns is about 0 (|k| < 0.15 for 20k days)	200	0
load_prices refuses a broken file with a clear ValueError/FileNotFoundError	10	0
market files end on 2026-10-30 (as documented)	3	0
market files load: finite returns, sorted unique dates	3	0
median_overshoot: median of peak-(placebo+1), inside its 95% interval	1,000	0
minute_to_step: within half a step of minute/390/dt and between 0 and steps/day	4,000	0
model.check accepts exactly the valid settings	20,000	0
model.check raises only ValueError on bad settings	20,000	0
no breakers -> no halts	2,821	0
no chartists, noise or news -> price and value stay exactly at 1	500	0
no chartists: z = 0 and x = 0 all along	801	0
no-herding regime (eps >> (1-delta)/(N-1)): 50/50 is the most likely	632	0
numba and pure-Python runs agree for the same seed	60	0
opinion x is between -1 and 1	3,880	0
optimist counts are whole numbers	3,880	0
paired_difference: mean of on-off, inside its own 95% interval	1,000	0
paper_loss = squared gaps in units of tolerance + kurtosis penalty	400	0
party_gap matches a direct computation	500	0
party_gap refuses mismatched or too-short input	2	0
pause_steps: at least 1 step, the nearest whole number of steps	4,000	0

percentile equals scipy's percentileofscore(kind='mean')	5,000	0
percentile refuses NaN / not-1-D input instead of a silent number	3	0
r is the day-to-day change of log price	4,681	0
returns() refuses an unknown market	1	0
same seed -> identical run	4,681	0
saved settings have step_pull below 1 (time step fine enough)	2	0
saved settings load and pass model.check	2	0
scaled_breakers multiplies every level by model_sd/market_sd and stays valid	1,992	0
scaled_breakers refuses levels of 100% or more	1,008	0
scaled_breakers refuses non-positive / NaN volatility	5	0
series lengths are days+1 (closes) and days (returns, halts)	4,681	0
simulate rejects bad arguments with ValueError	24	0
simulate returns or raises RunawayError (nothing else)	6,000	0
spx_loss = mean squared gap of medians in units of tol	400	0
standardize gives mean 0 and sd 1	2,845	0
starts at price = value = 1 (log 0)	4,681	0
stats functions reject bad arguments with ValueError	23	0
stats.acf: never returns NaN/inf silently	2,142	0
stats.acf: rejects NaN/inf	100	0
stats.acf: returns a value or raises ValueError (never another error)	4,000	0
stats.aggregate: never returns NaN/inf silently	2,567	0
stats.aggregate: rejects NaN/inf	100	0
stats.aggregate: returns a value or raises ValueError (never another error)	4,000	0
stats.alpha_regression: never returns NaN/inf silently	1,183	0
stats.alpha_regression: rejects NaN/inf	100	0
stats.alpha_regression: returns a value or raises ValueError (never another error)	4,000	0
stats.ccdf: never returns NaN/inf silently	2,845	0
stats.ccdf: rejects NaN/inf	100	0
stats.ccdf: returns a value or raises ValueError (never another error)	4,000	0
stats.fingerprint: never returns NaN/inf silently	855	0
stats.fingerprint: rejects NaN/inf	100	0
stats.fingerprint: returns a value or raises ValueError (never another error)	4,000	0
stats.hill_alpha: never returns NaN/inf silently	1,728	0
stats.hill_alpha: rejects NaN/inf	100	0
stats.hill_alpha: returns a value or raises ValueError (never another error)	4,000	0
stats.hurst: never returns NaN/inf silently	1,426	0
stats.hurst: rejects NaN/inf	100	0
stats.hurst: returns a value or raises ValueError (never another error)	4,000	0
stats.kurtosis: never returns NaN/inf silently	2,845	0
stats.kurtosis: rejects NaN/inf	100	0
stats.kurtosis: returns a value or raises ValueError (never another error)	4,000	0

stats.standardize: never returns NaN/inf silently	2,845	0
stats.standardize: rejects NaN/inf	100	0
stats.standardize: returns a value or raises ValueError (never another error)	4,000	0
stock_returns() refuses an unknown ticker	1	0
switches finds the start and each change	500	0
terms with min_days=0 does not crash on presidencies outside the dates	1	0
terms: in date order, no overlap, every term has >= 20 days	1	0
typical picks the history with the median mismatch	2,000	0
without news the value moves exactly by the policy shocks	490	0
worst_fall equals the brute-force worst k-day fall	5,000	0
worst_fall rejects bad input with ValueError	6	0

Sweep 2: realistic simulations (both fitted settings, seeds 50,000-50,399)

Property checked	Passed	Failed
_leave: number leaving matches Binomial(n, 1-exp(-(a+b)dt)) (5-sigma)	2,000	0
_leave: share going to A matches a/(a+b) (5-sigma)	2,000	0
a longer run starts with exactly the shorter run	800	0
breakers that can never fire (97-99%) leave the run unchanged	800	0
max_gap=None leaves a run that never ran away unchanged	800	0
no chartists: daily returns are close to a bell curve (|excess kurtosis| < 0.6 over 1,500 days)	800	0
shocks of 0% leave the run unchanged	800	0
with breakers, the run is identical to the run without until the first halt day	800	0

6. Where I would not yet bet $1M

- **Scientific conclusions are not proven by these tests.** The tests show the code does what SPEC.md says. Whether the model explains crashes is a scientific claim, and RESULTS.md already lists its limits (H of returns 0.39 vs 0.50; paper-matched results change with the time step).
- **Calibration was not rerun in full.** A full fit takes many hours (your notes say to run it overnight). Its pieces were tested (search ranges, loss functions, the resume-after-stop files, loading the saved settings), but not a complete fit from start to finish.
- **Lux & Marchesi (2000) parameter set I is still unverified** (SPEC.md says so too); it needs the CMU library copy.
- **Plots are checked by their tests and by being redrawn identically, not judged by eye in this audit.**
- The numba version was compared with plain Python on 60 short runs, not on long ones (plain Python is about 50x slower).

7. How to rerun

```
python3 -m pytest tests/ -q                # the project's 401 tests (~1.5 min)
python3 scratch/test_audit/fuzz.py         # sweep 1 (~20 s); scale with e.g. fuzz.py 0.1
python3 scratch/test_audit/fuzz2.py        # sweep 2 (~1 min)
zsh scratch/test_audit/run_scripts.sh      # every script (~0 min)
python3 scratch/test_audit/build_report.py  # this PDF
```

All audit files are in `scratch/`, which git ignores. To keep the sweeps, they could move into `tests/`.